DLDCA Outlab: Feeling the C Breeze

Week 5

September 8, 2026

Introduction

Now that we have a basic understanding of functions in assembly, lets take it to 11 by calling some C functions into the mix! `libc` works strictly within calling conventions and may modify caller-saved registers a lot. Since we need `libc`, we will now use `gcc` to link the binaries.

Necessary tools

We shall be using `nasm` to assemble x86-64 assembly into object files, and `gcc/g++` to link those object files into an executable. Before starting the lab, ensure both are installed:

```
$ nasm -v
NASM version [version]
$ gcc --version # or g++ --version
gcc ([linux-flavour] [version]-[linux-build]) [version]
```

Structure of submission/templates

Submission format:

```
your-roll-no/
|- task1/
    |- prestack.s (TODO)
|- task2a/
    |- read_gcd.s (TODO)
|- task2b/
    |- scanf_gcd.s (TODO)
|- task3/
    |- cat.s (TODO)
|- task4/
    |- time.s (TODO)
|- task5/
    |- hanoi.s (TODO)
|- task6/
    |- higher_order.s (TODO)
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz`. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Background

Note about stack alignment

The stack must be 16-byte aligned before you `call` into libc. Your own hand-written functions never needed to care about this, but compiled library code does, internally. Misaligned stacks can lead to errors very easily.

Note on using `printf`

When calling `printf()` from assembly under the SystemV calling convention, the `al` register must be set to the number of vector (SSE) registers used to pass floating-point arguments, since `printf` is a variadic function and the callee has no other way of knowing how many floating-point arguments were passed. If no floating-point arguments are used, `al` should simply be set to `0`.

Tasks

Task 1: Is there stuff already on the stack?!

Let's experiment with whats there on the stack as we come into `_start`. You may know that the typical way to declare the `main` function in C/C++ is `int main(int argc, char** argv)`, but did you know that `argc` and `argv` came on the stack!

So, if the value of `rsp` at entry into `_start` is `x`, `qword [x]` contains `argc`, `qword [x+8]` has `argv[0]`, `qword [x+16]` has `argv[1]`, and so on until `qword [x+8*argc]` has the last element of `argv`. (It is also guaranteed that `qword [x+8*argc+8]` is `0`/`nullptr`)

Implement the following from scratch:

- `my_strlen(const char* str{rdi}) -> size_t{rax}`: length of a null-terminated string, not counting the null-terminator. (See `std::strlen` for implementation details)
- `my_atoi(const char* str{rdi}) -> int64_t{rax}`: parse a (possibly negative) decimal integer from the start of a string. (See `std::atol` for implementation details)
- `gcd(int64_t a{rdi}, int64_t b{rsi}) -> int64_t{rax}` using tail call recursion (Euclidean Algorithm).

In `_start`, check that `argc == 3` and print `argv[1]` as-is, using `my_strlen` and a `write` syscall. Then exit with `gcd(my_atoi(argv[1]), my_atoi(argv[2]))`.

Fill in all four functions and `_start` in `prestack.s`.

To run your code:

```
$ make task1 args="48 18"
48
Return Code: 6
```

Task 2a: ok so you want interaction now?

We saw above how to look at the command-line arguments passed to us, but what if we want to take input from the user as the program runs? For that we need more `syscall`s!

The `syscall` for this purpose is the `read` syscall. Console input exists as the file descriptor '`0`', which needs to be given to `read`.

Read a line of the form `"a b"` (two numbers separated by a single space) from stdin using *one* `read` syscall into a buffer, then parse both numbers out of that buffer. Finally, find the gcd of the two numbers and print it to the console using `printf`.

(Does it work if you manually `syscall exit` vs calling the `exit` function? Why or why not?)

Fill in `my_strtol_10`, `gcd`, and `_start` in `read_gcd.s`. See std::strtol for implementation details (take `base=10`). See above on how to call `printf` from assembly.

To run your code:

```
$ echo "48 18" | make task2a
gcd = 6
```

Task 2b: Let's make it C

Let's do the same thing as last time, except we use more C functions. Instead of manually calling the `read` syscall, we can use the C function `scanf`.

Replace the `read` syscall and calls to `my_atoi` with a single call to `scanf`!

Fill in `gcd` and `_start` in `scanf_gcd.s`.

To run your code:

```
$ echo "30 45" | make task2b
gcd = 15
```

Task 3: `cat` (Meow!)

It's time to write code which is actually useful. As you know, `cat` is a `bash` utility which con**cat**enates files. To use files, we need more `syscall`s: `open` and `close`. To implement, for each file in `argv[1:]`, `open` it, `read` it, `write` it to the console and then `close` the file. If the filename is given as '`-`', use `stdin` as your file (don't `open` it though)

Fill in `_start` in `cat.s`.

To run your code:

```
$ make task3 args="some_file.txt"
[contents of some_file.txt]

Return Code: 0
```

Task 4: Excuse me, what time is it?

Get the current time via the `time` syscall (number `201`). It takes a single argument: a pointer to also store the result at (or `NULL` if you only want it back in `rax`), and returns the current Unix timestamp (seconds since *the epoch*: 1st Jan 1970). Use the `localtime_r` function from `C` to parse this Unix timestamp to a more readable format and print it.

Fill in `_start` in `time.s`.

Does your code gives you the time shown by the computer clock?

If not, how much is it off by? Does that time have any significance?

To run your code:

```
$ make task4
07/09/2026 07:21:35

Return Code: 0
```

Task 5: Towers of Hanoi

Implement

```
towers_of_hanoi(const char* src{rdi}, const char* dest{rsi},
                const char* aux{rdx}, size_t num_discs{rcx})
```

which moves `num_discs` discs from pole `src` to pole `dest` (using `aux` as the spare pole) one disc at a time, never placing a larger disc on a smaller one. Each time you move a disc, print it with `printf("Move disk %d from %s to %s\n", ...)`.

`src`, `dest`, and `aux` are pole names, you can pass them straight through to `printf` as `%s` arguments.

Maintain a running count of total moves made through `move_count`, increment with every printed move. For `num_discs` discs, this should always come out to $2^{\texttt{num_discs}} - 1$.

Fill in `towers_of_hanoi` and `_start` in `hanoi.s`.

To run your code:

```
$ make task5
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Total moves: 7

Return Code: 7
```

Task 6: Funception

Implement two higher-order functions that each take a *function pointer* as one of their arguments. You can `call` through it (e.g. `call rdx`) instead of calling a fixed label.

- `is_sorted(int* data{rdi}, size_t count{rsi}, FUNC cmp_func{rdx}) -> eax` checks whether the `int` array `data` is sorted according to `cmp_func`, where `cmp_func(a, b)` returns true if `a` is allowed to come before `b`. An array with 0 or 1 elements is trivially sorted.
- `count_if(int* data{rdi}, size_t count{rsi}, FUNC pred{rdx}) -> rax`: counts how many elements of `data` satisfy `pred(x)`.

Both `cmp_func` and `pred` take and return plain `int`s (32-bit), so use the 32-bit register names (`edi`, `esi`, `eax`, ...) when setting up and reading their arguments.

Write two comparators (`le` and `ge`, for ascending and descending) and one predicate (`is_positive`), then in `_start`, call `is_sorted` with each comparator and `count_if` with the predicate, all on the same array. `printf` all three results, then exit with
$100 \cdot$ `is_sorted(le)` $+ 10 \cdot$ `is_sorted(ge)` $+$ `count_if(is_positive)`.

Fill in `le`, `ge`, `is_positive`, `is_sorted`, `count_if`, and `_start` in `higher_order.s`.

To run your code:

```
$ make task6
is_sorted(le)=1 is_sorted(ge)=0 count_if(pos)=5

Return Code: 105
```

Table 1: Some basic x86-64 Assembly Instructions

Instruction	Operands	Effect
`mov`	`reg, reg/*mem/imm`	`reg` ← `reg/*mem/imm`
`mov`	`*mem, reg`	`*mem` ← `reg`
`movzx / movsx`	`reg, reg/*mem`	`reg` ← `reg/*mem`, zero-/sign-extended
`lea`	`reg, *mem`	`reg` ← address of `*mem`
`add`	`reg, reg/*mem/imm`	`reg` ← `reg` + `reg/*mem/imm`
`add`	`*mem, reg/imm`	`*mem` ← `*mem` + `reg/imm`
`sub`	`reg, reg/*mem/imm`	`reg` ← `reg` − `reg/*mem/imm`
`sub`	`*mem, reg/imm`	`*mem` ← `*mem` − `reg/imm`
`inc`	`reg/*mem`	`reg/*mem` ← `reg/*mem` + 1
`dec`	`reg/*mem`	`reg/*mem` ← `reg/*mem` − 1
`neg`	`reg/*mem`	`reg/*mem` ← −`reg/*mem` (Two's complement)
`not`	`reg/*mem`	`reg/*mem` ← ¬`reg/*mem` (One's complement/bitwise)
`shl / shr`	`reg/*mem, imm/cl`	Logical shift left / right by `imm` or `cl` bits
`sar`	`reg/*mem, imm/cl`	Arithmetic shift right (preserves sign bit)
`imul`	`reg, reg/*mem/imm`	`reg` ← `reg` × `reg/*mem/imm`
`imul`	`*mem, reg/imm`	`*mem` ← `*mem` × `reg/imm`
`div`	`reg`	`rax` ← `(rdx:rax)`/`reg` `rdx` ← `(rdx:rax)`%`reg`
`and`	`reg, reg/*mem/imm`	`reg` ← `reg` ∧ `reg/*mem/imm`
`and`	`*mem, reg/imm`	`*mem` ← `*mem` ∧ `reg/imm`
`or`	`reg, reg/*mem/imm`	`reg` ← `reg` ∨ `reg/*mem/imm`
`or`	`*mem, reg/imm`	`*mem` ← `*mem` ∨ `reg/imm`
`xor`	`reg, reg/*mem/imm`	`reg` ← `reg` ⊕ `reg/*mem/imm`
`xor`	`*mem, reg/imm`	`*mem` ← `*mem` ⊕ `reg/imm`
`cmp`	`reg/*mem, reg/imm`	Sets flags based on subtraction: `reg/*mem` - `reg/imm`. ZF = 1 if equal, SF = 1 if result negative, CF = 1 if borrow (unsigned), OF = 1 if signed overflow.
`test`	`reg/*mem, reg/imm`	Sets flags based on bitwise AND: `reg/*mem` ∧ `reg/imm`. ZF = 1 if result is zero, SF = 1 if high bit set, CF and OF are cleared. No result is stored.
`jmp`	`label`	Unconditional jump to label
`je / jz`	`label`	Jump if Zero Flag (ZF) = 1
`jne / jnz`	`label`	Jump if Zero Flag (ZF) = 0
`jl`	`label`	Jump if Less (SF ≠ OF)
`jle`	`label`	Jump if Less or Equal (ZF = 1 or SF ≠ OF)
`jg`	`label`	Jump if Greater (ZF = 0 and SF = OF)
`jge`	`label`	Jump if Greater or Equal (SF = OF)
`cmov`*cc*	`reg, reg/*mem`	`reg` ← `reg/*mem` if *cc* holds, else unchanged.
`set`*cc*	`reg/*mem`	`reg/*mem` ← 1 if *cc* holds, else 0.
`syscall`		Performs a syscall with syscall number stored in `rax` Arguments passed in other registers starting from `rdi`
`push`	`reg`	Pushes the value in `reg` onto the stack
`pop`	`reg`	Pops the value on top of the stack onto `reg`
`call`	`label`	Calls the function called `label`
`ret`		Pops the return address from stack and jumps to it

Table 2: Scalar Floating-Point (`xmm`) Instructions

Instruction	Operands	Effect
`movsd`	`xmm, xmm/*mem`	`xmm` ← `xmm/*mem` (low 64 bits)
`movsd`	`*mem, xmm`	`*mem` ← `xmm` (low 64 bits)
`movq`	`xmm, reg/*mem`	`xmm` ← `reg/*mem` (raw 64 bits, no conversion)
`movq`	`reg/*mem, xmm`	`reg/*mem` ← `xmm` (raw 64 bits, no conversion)
`xorps / xorpd`	`xmm, xmm`	`xmm` ← `xmm` $\oplus$ `xmm` (idiom to zero an `xmm` register)
`pxor`	`xmm, xmm`	`xmm` ← `xmm` $\oplus$ `xmm` (bitwise XOR on `xmm`)
`addsd`	`xmm, xmm/*mem`	`xmm` ← `xmm` + `xmm/*mem`
`subsd`	`xmm, xmm/*mem`	`xmm` ← `xmm` − `xmm/*mem`
`mulsd`	`xmm, xmm/*mem`	`xmm` ← `xmm` × `xmm/*mem`
`divsd`	`xmm, xmm/*mem`	`xmm` ← `xmm` ÷ `xmm/*mem`
`comisd`	`xmm, xmm/*mem`	Sets flags like `cmp`, comparing as doubles (use `ja/jae/jb/jbe`). Raises exception on NaN.
`ucomisd`	`xmm, xmm/*mem`	Same as `comisd`, but does not raise exception on NaN
`cvttsd2si`	`reg, xmm/*mem`	`reg` ← integer part, truncated toward zero
`cvtsi2sd`[1]	`xmm, reg/*mem`	`xmm` ← `reg/*mem` converted to a double

Table 3: Syscalls used in this outlab

Syscall	`rax`	Arguments (`rdi, rsi, rdx, ...`)	Returns (`rax`)
`read`	0	`fd, buf, count`	bytes read (0 = EOF)
`write`	1	`fd, buf, count`	bytes written
`open`	2	`path, flags, mode`	fd (`flags = 0` for read-only)
`close`	3	`fd`	0 on success
`time`	201	`ptr` (or `NULL`)	current Unix timestamp
`exit`	60	`status`	Does not return

[1]Prefer loading floating point literals or the binary representation directly instead of using this. Use this only for runtime `int` to `double` conversions